

BITS, PILANI – K. K. BIRLA GOA CAMPUS

Theory of Computation

(CS F51)

Lecture No. 16



ReCap - Myhill-Nerode Theorem

Myhill-Nerode Theorem

Let $L \subseteq \Sigma^*$ be a *regular* language. Then there exists DFA with *precisely as many states* as there are *equivalence classes* in $\approx_L$ that accepts L .

Proof: We define DFA $M = (K, \Sigma, \delta, s, F)$ as follows:

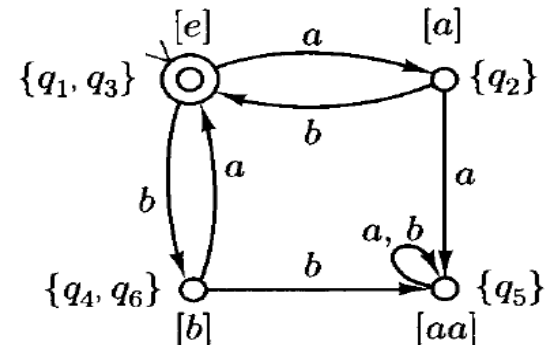
$K = \{[x] : x \in \Sigma^*\}$, the set of equivalence classes in $\approx_L$

$s = [e]$, the equivalence classes of *empty string* e in $\approx_L$

$F = \{[x] : x \in L\}$

Minimal DFA corresponding to $L = (ab \cup ba)^*$

For any $[x] \in K$ and any $a \in \Sigma$,
define $\delta([x], a) = [xa]$



Characterization of Regular Languages

Corollary:

A language L is regular if and only if $\approx_L$ has finitely many equivalence classes.

Algorithm for constructing minimal DFA

Algorithm for constructing **minimal DFA**

Given DFA M and L such that $L = L(M)$.

We need to compute equivalence classes of $\approx_L$

Let $M = (K, \Sigma, \delta, s, F)$.

Define a relation $A_M \subseteq K \times \Sigma^*$ as follows:

$(q, w) \in A_M \Leftrightarrow (q, w) \vdash_M^* (f, e)$ for some $f \in F$

Intuitively, $(q, w) \in A_M$ means that w drives M from q to an accepting state.

We say two states $q, p \in K$ are **equivalent** denoted by $q \equiv p$

if for all $z \in \Sigma^*$: $(q, z) \in A_M \Leftrightarrow (p, z) \in A_M$

Note: p & q are to be merged.

Algorithm for constructing minimal DFA

Our algorithm will compute $\equiv$ as the limit of a sequence of equivalence relations $\equiv_0, \equiv_1, \equiv_2, \dots$

For Two States $q, p \in K$

$q \equiv_n p$ if the following is true:

$(q, z) \in A_M \iff (p, z) \in A_M$ for all strings z with $|z| \leq n$

Observe:

- Each equivalence relation in $\equiv_0, \equiv_1, \equiv_2, \dots$ is a *refinement* of the previous one.
- $q \equiv_0 p$ holds if q & p are either both *accepting* or both *non-accepting*

So, there are exactly two equivalence classes of $\equiv_0$, F & $K - F$

We see how to find $\equiv_n$ from $\equiv_{n-1}$

Algorithm for constructing minimal DFA

Lemma: For any two states $q, p \in K$ & $n \geq 1$,

$q \equiv_n p \iff$ (a) $q \equiv_{n-1} p$ & (b) for $a \in \Sigma$, $\delta(q, a) \equiv_{n-1} \delta(p, a)$

Note: Lemma gives a method to *compute* equivalence classes of $q \equiv_n p$ by using the *already found* equivalence classes of $q \equiv_{n-1} p$

We *continue* until $\equiv_n$ is same as $\equiv_{n-1}$

Question: For being a algorithm, process should *terminate*.

For *each iteration* where $\equiv_n \neq \equiv_{n-1}$ we know $\equiv_n$ is a *proper refinement* of $\equiv_{n-1}$ & thus has at least one more equivalence class than $\equiv_{n-1}$.

But number of equivalence classes cannot be more than the number of states of M .

$\therefore$ algorithm will *terminate* after atmost $|K|-1$ steps

Algorithm for constructing minimal DFA

Example:

Initially equivalence classes

of $\equiv_0$ are

$\{q_1, q_3\}$ and $\{q_2, q_4, q_5, q_6\}$

After the *1st iteration* equivalence classes of $\equiv_1$ are

$\{q_1, q_3\}$, $\{q_2\}$, $\{q_4, q_6\}$ and $\{q_5\}$

Split happened because $\delta(q_2, b) \not\equiv_0 \delta(q_4, b)$

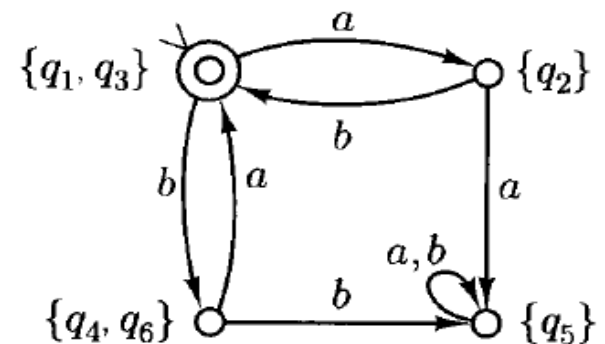
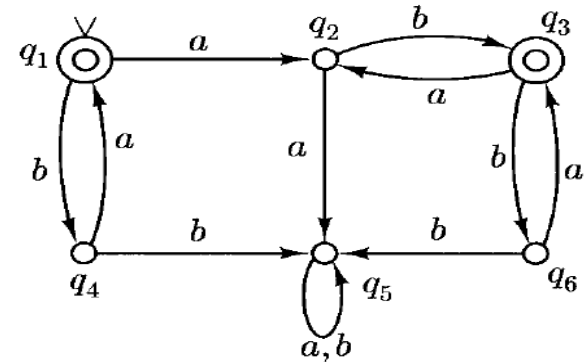
$\delta(q_2, b) \not\equiv_0 \delta(q_5, b)$ and $\delta(q_4, a) \not\equiv_0 \delta(q_5, a)$

After the *2nd iteration* there is *no*

splitting of classes & so

algorithm *terminates*.

The resulting *minimal* state DFA is:



Algorithm for constructing minimal DFA

Example:

We have one *unreachable* state, state D. We *remove* this state

$\equiv_0 \{C\}, \{A,B,E,F,G,H\}$

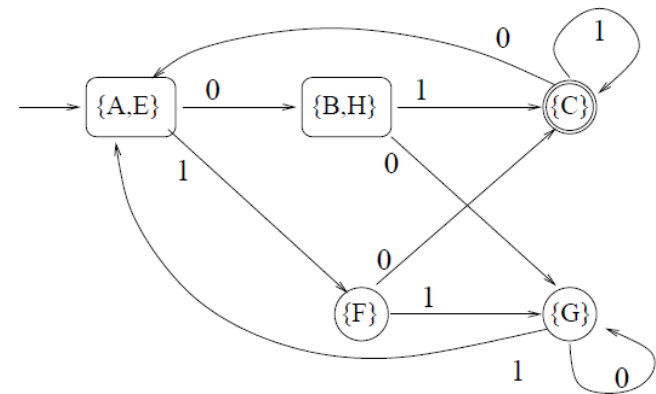
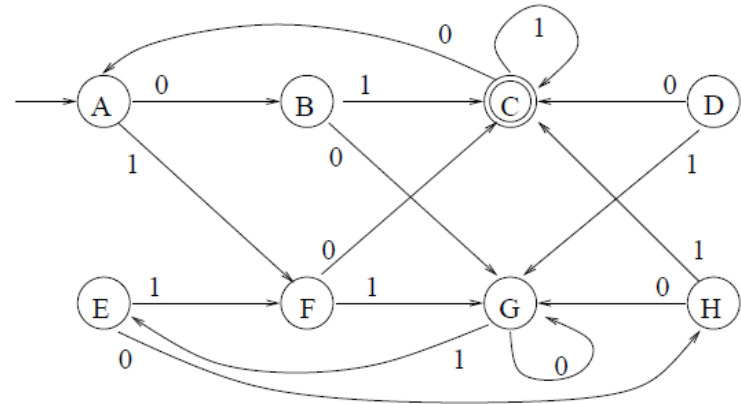
$\equiv_1 \{C\}, \{A,E,G\}, \{B,H\}, \{F\}$

$\equiv_2 \{C\}, \{A,E\}, \{B,H\}, \{G\}, \{F\}$

After the *3rd iteration* there is *no splitting* of classes

So algorithm *terminates*.

The resulting *minimal* state DFA is:



Pumping Lemma

Pumping Lemma *cannot show* a language is *regular*

Let $L = \{a^i b^j c^j : i \geq 1 \text{ \& } j \geq 0\} \cup \{b^j c^k : j, k \geq 0\}$

Then L is *not regular*

But the conclusion of Pumping Lemma is true.

Exercise